Design Rationale for Payment

Object-Oriented Design Principles

Two classes GooglePay and ApplePay implement the Payment interface.

The payment methods all exist for the purpose of adding money to the user's balance. This means that they are only responsible for one part of the program's functionality. (Single Responsibility Principle)

Payment is an interface which is implemented by GooglePay and ApplePay. Since Payment is an interface, there is no need to modify Payment when adding a new payment method. This keeps Payment open for extension but closed for modification. (Open-closed Principle)

AddBalanceAction class method has a parameter that takes in a Payment object. Since GooglePay and ApplePay implement Payment, they can be used in place of Payment without disrupting the behaviour of the program. (Liskov Substitution Principle)

Classes that use payment functionality, or high-level modules, depend on the Payment interface instead of GooglePay or ApplePay, which are concrete implementations. This makes it easy to swap payment methods. Furthermore, Payment does not depend on the details of the concrete implementations. It instead defines the contract for processpayment and leaves the specific implementation to the classes that implement it. (Dependency Inversion Principle)

Why not switch-case

Switch-case should not be used as it violates the object-oriented design principles.

It violates OCP as adding a new payment method would require modifying the existing switch statement.

It also violates DIP as it depends entirely on concrete implementations. This makes it harder to extend the program without modifying the class.

Justification

Making Payment an abstract class was considered but the idea was ultimately scrapped.

Abstract class:

Pros:

- Shared implementation reduces code duplication.
- Adding new methods to the abstract class will not break the existing subclasses.

Cons:

- Does not allow for multiple inheritance.
- Hard to change the hierarchy once established.
- Cannot achieve polymorphism.
- Less clear contract makes it harder to make new subclasses.

Interface:

Pros:

- Each payment method can define its own behaviour.
- Enables polymorphism which improves extensibility and maintainability.
- Clear contract for implementation which enhances readability and maintainability.

Cons:

- No shared implementation means the classes need to implement all methods in the interface which could lead to duplicate code.
- Adding new methods to the interface will break the subclasses as they need to implement all methods of the interface.

After weighing the pros and cons, the reason abstract class was not chosen is because it would require the payment methods to inherit the common behaviours. This makes it difficult to deal with payment methods that should behave differently.

Interface was chosen as it allows each payment method to have its own implementation just like the LIMIT placed on adding balance in ApplePay. This allows the subclasses to behave entirely differently while adhering to the methods defined in the interface. Furthermore, the lack of shared implementation and difficulty in adding new methods are not causes for concern as each payment method would behave differently and the only method needed is the ability to process payments.